Variable Table Editor

📋 Important Notice

This assignment and related code are confidential information. Please handle with care.

- ⚠️ **Strictly Prohibited from Sharing**: Do not share the assignment, source code, or any related information with others
- ⚠️ **Prohibited from Distribution**: Do not publish or discuss on public platforms (GitHub, blogs, forums, etc.)
- ⚠️ **Confidentiality Agreement**: Participation in the interview indicates agreement to confidentiality terms

Background

We are developing an automation programming platform that requires a **Variable Table Editor** to manage variable definitions in control system programs. This interview assignment simulates the core functionality of this scenario.

Requirements Description

Implement a simplified version of a variable table editor that supports CRUD operations for variables and import/export in standard text format (VAR...END_VAR).

Core Features

1. **Table Management**: Display variable list with name, data type, default value, comments, etc.
2. **Data Operations**: Support adding rows, deleting rows, editing cell content
3. **Data Validation**: Name uniqueness validation, data type and default value format validation
4. **Format Conversion**: Support import and export in standard text format (VAR...END_VAR)

Technical Requirements

- Mandatory
  - Use React + TypeScript

    - Use a state management solution (Mobx, Redux, Zustand, etc.)
    - Cover all ACs (Acceptance Criteria) listed below
    - Complete unit tests based on ACs
    - Provide basic error prompts and handling
    - Use git for source code management, show feature completion history, recommended small commits
- Optional
    - Can use any table component library (such as ag-Grid, react-table, antd Table, etc.)
    - Can use any CSS framework and component library
    - Choose unit testing framework yourself
    - No requirements for needs not mentioned in ACs
    - Can use AI tools for assisted coding, but total code should not exceed 50%
- Bonus Points
    - Clear code structure with distinct component responsibilities
    - Integration tests and end-to-end tests are bonus points
    - Provide CI/CD implementation
    - Use object-oriented programming design
    - Test-driven development (please commit in small steps to demonstrate the TDD process)

Deliverables

1. **Source Code**
    i. Complete runnable project source code (delete node_modules etc. to save space)
    ii. Include git repository and commit records to show development process
    iii. If AI assisted development, please submit AI usage records
2. **README.md**: Including
    i. How to run the project
    ii. Project structure description
3. **Design Document**: Briefly explain design ideas and technology selection
4. Please package the above deliverables into a compressed file, then send an email to SiCong.He@siemens.com and QiuYang.Chen@siemens.com. After receiving, we will reply to confirm receipt. If you do not receive an email reply within two days, it may have been filtered by the mailbox. Please write a separate email to the above mailbox, and we will tell you another way to handle it.

AC 1: Table Display

Given Open the variable table editor page

When Page loading is complete

Then

- Display an empty table with the following columns: Index, Name, Data Type, Default Value, Comment
- Index column is read-only, automatically generated
- Display "Add Row" and "Delete Row" buttons
- Display text input area and "Import", "Export" buttons

AC 2: Add Variable Row

Given Table is currently empty or has data

When User clicks "Add Row" button

Then

- Add a new row at the end of the table
- All fields of the new row (name, data type, default value, comment) default to empty
- Index is automatically generated as current maximum index + 1

AC 3: Delete Variable Row

Given Table has at least one row of data

When User selects a row and clicks "Delete Row" button

Then

- The row is deleted from the table
- Subsequent row indices are automatically recalculated

AC 4: Edit Variable Name

Given Table has data rows

When User clicks the name cell and enters a new name

Then

- If input is empty, restore to original value, show error prompt
- If the entered name already exists in the table (case-insensitive), show "Name already exists" error prompt, not allowing save
- If the entered name is unique and non-empty, save successfully

Example:

- Table already has `counter`
- User enters `Counter` → show error prompt
- User enters `newCounter` → save successfully

AC 5: Select Data Type

Given Table has data rows

When User clicks the data type cell

Then

- Show dropdown selection box with two options: `BOOL` and `INT`
- After user selects data type, cell displays the selected type

And When User switches data type

Then

- Default value cell automatically updates to default value of new type (BOOL → `TRUE`, INT → `0`)

Example:

- Current type is `BOOL`, default value is `TRUE`
- User changes type to `INT` → default value automatically changes to `0`

AC 6: Edit BOOL Type Default Value

Given Variable row with data type `BOOL`

When User clicks default value cell and enters value

Then

- Accept inputs: `true`, `false`, `TRUE`, `FALSE` (case-insensitive)
- Display format unified to uppercase: `TRUE` or `FALSE`
- If other values are entered, show error prompt

Example:

- User enters `TRUE` → display `TRUE`
- User enters `false` → display `FALSE`
- User enters `yes` → show error prompt

AC 7: Edit INT Type Default Value

Given Variable row with data type `INT`

When User clicks default value cell and enters value

Then

- Accept input: integers, range `-2147483648` to `2147483647`
- If non-integer or out of range, show error prompt

Example:

- User enters `42` → save successfully
- User enters `3.14` → show error prompt
- User enters `9999999999` → show error prompt (out of range)

AC 8: Edit Comment

Given Table has data rows

When User clicks comment cell and enters text

Then

- Accept any text input
- Can be empty

AC 9: Import Standard Text Format Variables

Given Table is currently empty or has data

When User pastes text in the following format in the text input box:

```
VAR
    isReady : BOOL := TRUE; // System ready flag
    counter : INT := 0; // Counter
    temperature : INT;
END_VAR
```

And Clicks "Import" button

Then

- Parse the text, extract variable names, data types, default values, and comments
- Clear existing data in the table, add parsed variables to the table
- If no default value, automatically fill default value based on data type (BOOL → `TRUE` , INT → `0`)
- Data types are case-insensitive (`BOOL` , `Bool` , `bool` are all recognized as `BOOL`)
- BOOL type default values are case-insensitive, display unified as uppercase (`TRUE` , `FALSE`)

Example:

- Input text contains `isReady : BOOL := true; // comment` → table displays `TRUE`
- Input text contains `isReady : BOOL := FALSE; // comment` → table displays `FALSE`
- Input text contains `temperature : INT;` (no default value) → table automatically fills `0`
- Table adds rows: name= `isReady` , type= `BOOL` , default value= `TRUE` , comment= `comment`

AC 10: Import Error Handling

Given User pastes standard text format (VAR...END_VAR) text in the text input box

When Text format cannot be parsed or contains unsupported data types

Then

- Show error prompt explaining specific error reason
- Do not add any data to the table

Example:

- Input `isReady : String := "test";` → show "Unsupported data type: String"
- Input `isReady Bool false` → show "Format error, cannot parse"

AC 11: Export to Standard Text Format

Given Table already has the following data:

Index	Name	Data Type	Default Value	Comment
1	isReady	BOOL	TRUE	System ready flag
2	counter	INT	0	Counter
3	temperature	INT		

When User clicks "Export" button

Then

- Display text in the following format in the text input box:

```
VAR
    isReady : BOOL := TRUE; // System ready flag
    counter : INT := 0; // Counter
    temperature : INT;
END_VAR
```

- Data types use unified uppercase format (`BOOL` , `INT`)

- BOOL type default values use unified uppercase format (`TRUE` , `FALSE`)
- Variables without default values do not include `:=` part
- Variables without comments do not include `//` part